Logs are timestamped and should be structured, ideally including some context (e.g., labels) that can be used for correlation with other signals. Usually, humans consume logs, either directly, by searching for an event, or indirectly, via correlation.

> **NOTE** There are many ways to refer to a single log event, including log lines, log items, log entries, and perhaps others I have not come across yet. To keep things simple, we will consistently use the term *event* here, but these terms are, by and large, interchangeable.

In the context of logs, we usually differentiate between *levels*, which can represent the nature of a log event. Different programming languages, libraries, and frameworks have, over time, established conventions; however, the following are some common semantics:

- Fine-grained logs events that are relevant for developing or fixing a bug (`DEBUG`)
- Production-type log events, including the following:
  - Expected behavior (usually `INFO`)
  - Unexpected behavior, such as `WARNING` or `ERROR`

There are several log standards and formats available, including line-oriented formats, such as syslog RFC 5424 (https://tools.ietf.org/html/rfc5424) and the Apache common log format (CLF; http://mng.bz/lWg8); JSON log formats (supported by most modern libraries); and binary formats, such as the journal file format (http://mng.bz/Bmw8), found in the context of `systemd`. To learn more, check out Graylog's Logs Format Guide (http://mng.bz/d1nO). Some good practices in the context of logs include the following:

- Consider what to log, with the goal of providing actionable insights (context is king).
- Log to `stdout` rather than to a fixed location, such as a file. This enables log agents to collect and route your log events. See also the entry in the 12-factor methodology (https://12factor.net/logs) for some background.
- Use labels to enhance the value of a log event. For example, you can use the component name or type, the kind of source, or the type of operation as labels to aid filtering for events.

> **TIP** If you're wondering how you can "future proof" your strategies around logs, I recommend reading Ben Sigelman's (cofounder of Lightstep, known for his work on OpenTelemetry, Google Dapper, and Monarch) 2001 Twitter thread, "Why Tracing Might Replace (Almost) All Logging" (http://mng.bz/rWzD), in which he argues that in cloud-native systems, tracing will replace logging for the brunt of the workload via instrumentation about the transactions themselves (as opposed to logging about infrastructure or other resources). This makes sense, at least for a large number of use cases for logs in a transactional setup, and I want to encourage you to consider this kind of thinking and apply it to your environment, if you're in a position to do so. Naturally, many environments are more on the brown field than green field